9 APPENDIX

9.1 Pseudo-code of VPL Index Construction

The pseudo-code of *VPL* construction is presented in Algorithm 3. Specifically, we adopt *PUNCH* [38] as the graph partitioning method to divide G into multiple subgraphs $\{G_i|1 \leq i \leq k\}$ (Line 1), because it generates the minimal boundary vertex number on road networks [49] and its partition result complies well with the query clusters. Given the partition result, we first derive its order r via *tree height-aware PSP ordering* (will be introduced in Section 5.2). A tree decomposition T and the *shortcut arrays* of its tree nodes are then built based on r (Line 3). As for the label construction, we first build the overlay index for boundary vertices from the root of T (Lines 4-5) and then construct the labels for all partitions' non-boundary vertices in parallel (Lines 6-8). For each tree node $X(v)$, we first obtain the position array by identifying the position of any vertex $x_j \in X(v)$ in $X(v).A$ (Lines 12-13). The distance array is computed by using $X(v).N$ as the vertex separator, while the boundary array is computed by inheriting corresponding label entries in $X(v).dis$.

Algorithm 3: VPL Index Construction

```
   Input: Road network G
   Output: VPL index L = {X(v)|v ∈ V}
1  G̃, {G_i|i ∈ [1,k]} ← PUNCH(G);                    ▷ Obtain the partition result by PUNCH [38]
2  r ←PSPOrdering(G̃, {G_i});                          ▷ Tree Height-Aware PSP Vertex Order
3  T ←TDandShortcutBuild(G, r);                       ▷ TD and shortcut construction based on r
4  for all X(v) ∈ T, v ∈ G̃ in a top-down manner do
5      TreeNodeBuild(X(v));                           ▷ Overlay index construction
6  parallel_for G_i, i ∈ [1,k]
7      for all X(v) ∈ T, v ∈ G_i \ B_i in a top-down manner do
8          TreeNodeBuild(X(v));                       ▷ Partition index construction
9  return L;
10 Function TreeNodeBuild(X(v)):
11     Suppose v ∈ G_i and X(v) = X(v).N ∪ v = (x_1, x_2, ..., x_|X(v)|);
12     for j = 1 to |X(v)| do
13         X(v).pos[j] ←the position of x_j in X(v).A;
14     for j = 1 to |X(v).A| − 1 do
15         c ← X(v).A[j]; X(v).dis[j] ← ∞;
16         for k = 1 to |X(v).N| do
17             if X(v).pos[k] > j then d ← X(x_k).dis[j];
18             else d ← X(c).dis[X(v).pos[k]];
19             X(v).dis[j] ← min{X(v).dis[j], X(v).sc[k] + d};
20         if v ∉ B and c ∈ B_i then
21             X(v).disB[l] ← X(v).dis[j], where l is the position of c in B_i;
22     X(v).dis[|X(v).A|] ← 0;
```

9.2 Pseudo-code of DVPL Shortcut Maintenance

Algorithm 4 presents the pseudo-code of mixed shortcut maintenance. We use the non-partitioned L to demonstrate the key idea of *DVPL* while it is easy to extend it to a partitioned one with parallelization. In particular, we use a min-heap set Q to record affected tree nodes, and a min-heap set $S(a)$ to record neighbors of a' affected shortcuts, ensuring that the updates are processed in a bottom-up manner. When Q is not empty, we iteratively pop the vertex a with the minimum order from Q, and handle its affected shortcuts $sc(a,b), \forall b \in S[a]$ (Lines 2-8). For each affected shortcut, we compute its new value d_{sc} and supportive number ϕ_{sc} via *minimum weight property* [24] (Line 5). For example, as shown in Figure 10-(a), suppose $sc(a,b)$ has two supportive vertices v_1, v_2 and the edge $e(a,b)$ decreases from 4 to 3, we have $d_{sc} = 3$ and $\phi_{sc} = 1$ since only edge $e(a,b)$ makes new $sc(a,b)$ holds after update. We compare $sc(a,b)$ with d_{sc} to figure out its update types, and assign Cw with old $sc(a,b)$ or new d_{sc} for the increase and decrease (Lines 6-7). After refreshing $sc(a,b)$ and $\phi_{sc}(a,b)$, we propagate the update in function SCUpdatePropagate, where we first identify the affected labels and then detect other affected shortcuts. We next take the decrease as an example (Lines 10-14). We first identify the affected distance labels in function DecUpdateCheck, *i.e.,* for each $u \in X(a).A$, if $L(a,u) > Cw + L(b,u)$ or $L(a,u) > Cw + L(u,b)$, we say the label $L(a,u)$ is affected and use $C(a,u)$ to record this information, where $C(a,u) > 0$ indicates increase update while $C(a,u) < 0$ represents decrease update. We recognize $X(a)$ as affected if there is any $L(a,u)$ that is affected. We also set the boolean value $X(a).ifU$ as true if there is any $L(a,ut)$ that is affected. As illustrate in Figure 10-(b), $L(a,u_1)$ is affected since $Cw + L(u_1,b) = 3 + 2 = 5 < L(a,u_1)$ while $L(a,u_2)$ is unaffected. We then check whether each neighboring shortcut $sc(b,u), u \in X(b).N$ or $sc(u,b), b \in X(u).N$ is affected in function DecSCCheck, where $sc(u,b)$ is recorded as affected if new $dis < sc(u,b)$ (Lines 20-22).

Algorithm 4: DVPL: Mixed Shortcut Maintenance

```
   Input: G and its mixed updates U, L
   Output: Shortcut Updated L
1  Q.insert(a), S[a].insert(b), ∀e(a,b) ∈ U; //Q is a min-heap set, S[a] is a set for a
2  while Q is not empty do
3      a ← Q.begin(), Q.erase(a); //a has the minimum vertex order in Q
4      for each b ∈ S[a] do
5          Compute new shortcut d_sc and φ_sc by minimum weight property [24]
6          if d_sc > sc(a,b) then Cw ← sc(a,b); //increase update
7          else if d_sc < sc(a,b) then Cw ← d_sc; //decrease update
8          sc(a,b) ← d_sc; φ_sc(a,b) ← φ_sc; SCUpdatePropagate(a,b,Cw);
9  Function SCUpdatePropagate(a,b,Cw):
10     if sc(a,b) is decrease update then
11         DecLabelCheck(a,b,Cw);                     ▷ Identify affected labels
12         for each u ∈ X(a).N do
13             if u ∈ X(b).N then DecSCCheck(b,u,a,Cw + sc(a,u));
14             else if b ∈ X(u).N then DecSCCheck(u,b,a,Cw + sc(a,u));
15     else if sc(a,b) is increase update then
16         if X(a).FN[b] = true then IncLabelCheck(a,b,Cw);
17         for each u ∈ X(a).N do
18             if u ∈ X(b).N then IncSCCheck(b,u,Cw + sc(a,u));
19             else if b ∈ X(u).N then IncSCCheck(u,b,Cw + sc(a,u));
20 Function DecSCCheck(u,b,a,dis):
21     if dis = sc(u,b) and u ∉ S[a] then φ_sc(u,b) ← φ_sc(u,b) + 1;
22     else if dis < sc(u,b) then Q.insert(u), S[u].insert(b);
23 Function IncSCCheck(u,b,dis):
24     if dis = sc(u,b) then
25         φ_sc(u,b) ← φ_sc(u,b) − 1;
26         if φ_sc(u,b) < 1 then Q.insert(u), S[u].insert(b);
27 Function DecLabelCheck(a,b,dis):
28     if Cw = L(a,b) then φ_d(a,b) ← φ_d(a,b) + 1, X(a).FN[b] ← true;
29     else if Cw < L(a,b) then C(a,b) ← −1, X(a).ifU ← true;
30     for each u ∈ X(a).A do
31         if L(a,u) > Cw + L(b,u), r(u) > r(b) or
             L(a,u) > Cw + L(u,b), r(u) < r(b) then C(a,u) ← −1;
32 Function IncLabelCheck(a,b,Cw):
33     for each u ∈ X(a).A do
34         if L(a,u) = Cw + L(b,u), r(u) > r(b) or
             L(a,u) = Cw + L(u,b), r(u) < r(b) or u = b then
35             φ_d(a,u) ← φ_d(a,u) − 1;
36             if φ_d(a,u) < 1 then C(a,u) ← 1, X(a).ifU ← true;
```

9.3 Discussion on Concurrency Control

From the perspective of transaction management, if we view the path index (labels) L and the graph G as the data, then the distance query Q corresponds to the *read*, and the update U with index maintenance corresponds to the *write*. When the update comes, the graph can be switched almost instantly as the modification of edge weights is very lightweight, while the index takes a while to maintain. Therefore, traditionally, the index can be viewed as *locked* during maintenance, so the query has to be answered through searching the graph. When the index is updated, it can be *unlocked* for fast query processing. This *Two-Phase Lock (2PL)* paradigm guarantees the δt-freshness of query processing, but the system throughput suffers when the index is locked. On the other hand, *Multi-Version Concurrency Control (MVCC)* of the SP index can keep the throughput at the highest level by allowing the queries to use the previous version of the index. However, it violates the update-first queuing model adopted in this paper and thus the affected query during maintenance would have *"incorrect" results* (*i.e.,* the results are different from the ones on the latest graph snapshot). The proposed VPSP framework also follows the 2PL of the SP indexes but gradually resorts to the updated indexes for higher throughput via the multi-stage query processing. More importantly, it further explores the correctness of the *older version* of the SP index by identifying the unaffected queries in the index validation module. For the affected queries, the multi-stage pipeline locks all the indexes initially and unlocks the indexes one by one, such that the throughput grows back earlier while the accuracy is always guaranteed. Furthermore, the rotation schedule further reduces the affected query number such that more queries can be answered correctly and efficiently with the older index. We further discuss the relationship and key differences between our VPSP framework and traditional MVCC as follows.

MVCC is a foundational technique in database systems that enables high-throughput concurrent reads and writes without mutual blocking. Its core principle is to maintain multiple versions of data: write operations create new versions rather than overwriting existing ones, while read operations access a consistent snapshot of the data from a specific point in time. This design philosophy aligns closely with the goals of our work, as both approaches seek to maximize system throughput by minimizing interference between read and write operations.

The fundamental difference between our VPSP and traditional MVCC lies in their problem definitions and correctness criteria, which stem from the distinct requirements of their respective application domains.

VPSP Framework: 1) **Problem Definition**: We address the High-Throughput Shortest Path (HTSP) problem, which requires the system to maximize query throughput while guaranteeing that all query results are computed on the most recent δt-fresh snapshot of the road network [34, 37, 44]. 2) **Query Correctness**: To ensure δt-freshness, queries must return results based on the fully updated graph G_{latest}.

MVCC: 1) **Problem Definition**: MVCC is a classical concurrency control mechanism widely adopted in database systems to improve transaction concurrency by allowing reads and writes to proceed on different versions of data concurrently. 2) **Query Correctness**: Traditional MVCC adopts a snapshot-consistency model. Under snapshot isolation, each transaction observes a logically consistent snapshot of the database corresponding to a particular historical version. Therefore, query correctness is defined with respect to the selected snapshot rather than the latest database state. In the context of dynamic road networks, an MVCC-based solution for the SP index naturally allows shortest path queries to continue using an older index version while a newer version is being updated and maintained.

Thus, the most fundamental distinction between VPSP and MVCC is their definition of a "correct" query result. VPSP requires that all results reflect the **current state of the road network** (within the δt freshness window), while MVCC only requires that results reflect a **consistent snapshot of the database at some point in time**. This difference is not a matter of implementation detail but a fundamental requirement imposed by the application domain: location-based services cannot tolerate stale routing results that do not reflect current traffic conditions, even if those results are consistent with some past state of the network.

Complementary Relationship. It is important to emphasize that VPSP and traditional MVCC are complementary rather than competing paradigms. MVCC of SP index provides a general solution for concurrent read-write access, but it cannot guarantee δt-freshness in scenarios where index maintenance latency is non-negligible. However, this does not mean that older index versions are entirely useless. They can still correctly answer queries that are **unaffected** by the updates. A key contribution of our work is the theoretical analysis and practical algorithm that precisely identifies these unaffected queries, allowing us to safely leverage the old index version for high-performance processing while guaranteeing δt-freshness for all queries. In essence, VPSP can be viewed as a spatially-aware extension of MVCC that bridges the gap between the general-purpose MVCC model and the domain-specific requirements of high-throughput dynamic shortest path query processing.

9.4 Proofs

Proof of Lemma 1. There are three cases for the edge weight decrease update.

Case 1: $e(a,b) \in sp(s,t)$ before update. In this case, $e(a,b)$ must be in $sp'(s,t)$ after update, thus $d'(s,t) = d(s,a) + |e'(a,b)| + d(b,t) < d(s,t)$.

Case 2: $e(a,b) \notin sp(s,t)$ before update but is in $sp'(s,t)$ after update. In this case, we also have $d'(s,t) = d(s,a) + |e'(a,b)| + d(b,t) < d(s,t)$.

Case 3: $e(a,b) \notin sp(s,t)$ before update and is not in $sp'(s,t)$ after update. Then $d'(s,t) = d(s,a) + |e'(a,b)| + d(b,t) > d(s,t)$. □

Proof of Lemma 3. The lower bound of the number of affected queries is determined by *Case D1*, which is $\phi(a,b)$. *Case D2* determines the upper bound of the number of affected queries, which may affect all queries on G in the worst case. □

Proof of Lemma 4. As per the definitions of $\Omega_{\bar{A}}$, $L(s)$ and $L(t)$ are untouched during the index maintenance. Therefore, $Q(s,t)$ is an unaffected query and could be computed by the "old" index L. □

Proof of Lemma 5. $d(s,t) \cdot v_{max}$ is the longest spatial distance that $sp(s,t)$ could traverse since v_{max} is the maximum speed limit on G.

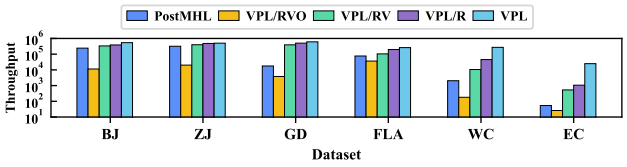


Figure 17: Ablation Study

Therefore, using $d(s,t) \cdot v_{max}$ as the major axis of the ellipse with foci at s and t is the largest spatial area $sp(s,t)$ can traverse. □

Proof of Theorem 2. We prove it by contradiction. Suppose G_j is inner-affected, there must $\exists sp'(s,t), s,t \in G_j$ that passes through at least one updated edge $e \in G_i$. We take the concise form of $sp'(s,t)$ as $sp'_c = \langle s, b_1, ..., b_q, t \rangle$. We have $b_1, b_q \in G_j$ and $sp'(b_1, b_q)$ passes through $e \in G_i$ after update. There are two update cases for the length change of $sp(b_1, b_q)$:

Case 1: suppose $d(b_1,b_q) > d'(b_1,b_q)$. $sp'(b_1,b_q)$ must pass through at least one decreased $e(a,b) \in G_i$ after update as per Lemma 1. However, $d'(b_1,b_q) = d'(b_1,a) + e'(a,b) + d'(a,b_q) > d(b_1,b_q)$ because $\Omega_{max}(G_j)$ does not overlap with the MBR of G_i, which is contradict with the assumption $d(b_1,b_q) > d'(b_1,b_q)$.

Case 2: suppose $d(b_1,b_q) < d'(b_1,b_q)$. $sp(b_1,b_q)$ must pass through at least one increased edge $e(a,b) \in G_i$ before update as per Case I1, which is contradict with the assumption $\Omega_{max}(G_j)$ does not overlap with the MBR of G_i. □

Proof of Theorem 3. No updates within G_i means that there is no inner change source, while $d(b_1,b_2), \forall b_1, b_2 \in B_i$ remain unchanged after indicates that there is no outer change source triggering the same-partition query. Therefore, G_i is inner-unaffected. □

Proof of Theorem 4. *Step 3* ensures the greedy PSP ordering is boundary-first. We prove the correctness of query processing as follows. Given query $Q(s,t), \forall s \in G_i, t \in G_j$, we take the concise form of $sp(s,t)$ with only OD pair and boundary vertices as $sp_c = \langle s, b_1, ..., b_q, t \rangle$, and prove the correctness in three cases.

Case 1: $s,t \in B$. $d(s,t)$ must be correctly computed since G_c preserves all shortest distances among boundary vertices.

Case 2: $s \notin B, t \in B$ (vice versa). Suppose u is the highest-order non-boundary vertex in $sp(s,d_1)$, there are two subcases. *SubCase 1*: $u = s$, we have $b_1 \in X(s).N$ since other vertex $v \in sp(s,b_1)$ is contracted before s, then b_1 is the ancestor of s and it naturally holds. *SubCase 2*: $u \neq s$, we have $b_1 \in X(u).N$ as per *SubCase 1* and $u \in X(s).A$ since $\forall v \in sp(s,u)$ must be contracted before u, forming the shortcut $sc(s,u)$. So $b_1 \in X(s).A$ and *Case 2* holds.

Case 3: $s,t \notin B$. There must exist $L(s,b_1)$ and $L(t,b_q)$ as per *Case 2*, and $d(b_1,b_q)$ is computed by *Case 1*. Hence, this case holds. □

Proof of Theorem 5. There are three cases for $sc(a,u)$ with $u \notin S[a]$. *Case 1*: $sc(a,u)$ decreases to $sc'(a,u)$. We have $sc'(a,b) + sc'(a,u) < sc(u,b)$ and $sc(u,b)$ needs to be maintained. Therefore, there is no need to increase $\phi_{sc}(u,b)$. *Case 2*: $sc(a,u)$ increases to $sc'(a,u)$. We have $sc'(a,b) + sc'(a,u) \neq sc(u,b)$, and thus there is no need to increase $\phi_{sc}(u,b)$. *Case 3*: $sc(a,u)$ remains unaffected. We have $sc'(a,b) + sc(a,u) = sc(u,b)$, and thus it is necessary to increase $\phi_{sc}(u,b)$ by 1. Otherwise, the increase update of $sc(u,b)$ may be incorrect in the next batch due to the wrong $\phi_{sc}(u,b)$. □

Proof of Theorem 6. $C(u,b) = 0$ means that $L(u,b)$ is unaffected. There are three cases for $L(u,b)$.

Case 1: $L(u,b)$ decreases to $L'(u,b)$. We have $L'(a,b) + L'(u,b) < L(u,a)$ and $L(u,a)$ needs to be maintained. Therefore, there is no need to increase $\phi_d(u,a)$.

Case 2: $L(u,b)$ increases to $L'(u,b)$. We have $L'(a,b) + L'(u,b) \neq L(u,a)$, and thus there is no need to increase $\phi_d(u,a)$.

Case 3: $L(u,b)$ remains unaffected. We have $L'(a,b) + L'(u,b) = L(u,a)$, and thus it is necessary to increase $\phi_d(u,a)$ by 1. Otherwise, the increase of $L(u,a)$ may be incorrect in the next batch due to the wrong $\phi_d(u,a)$. □

Proof of Theorem 7. Mixed label maintenance algorithm checks all affected labels with $O(\Delta_d^T \cdot \Delta h)$ while only dealing with the affected labels $O(\Delta_d)$ with the computation of new label $O(\omega)$, the update propagation $O(deg^{max}_{\mathcal{G}^-})$. Therefore, the complexity is $O(\Delta_d^T \cdot \Delta h + \Delta_d \cdot (\omega + deg^{max}_{\mathcal{G}^-}))$. The space complexity consists of four parts: 2-hop labels and ancestors $O(n \cdot h)$, shortcut size $O(n \cdot \omega)$, the supportive vertices for shortcut update $O(n \cdot \sup^{max}_{sc})$, and the downward neighbors $O(n \cdot deg^{max}_{\mathcal{G}^-})$. Therefore, the space complexity is $O(n \cdot (\omega + h + \sup^{max}_{sc} + deg^{max}_{\mathcal{G}^-}))$. □

9.5 Additional Experimental Results

Exp 7: Ablation Study. As shown in Figure 17, we test throughput by removing the certain components: **R** is the rotation update schedule, **V** is the unaffected validation, and **O** is the THP ordering. From left to right, it can be viewed as adapting the *PostMHL* by adding these components one by one. Firstly, *VPL/ROV* has lower throughput than *PostMHL*, because its order was the unoptimized naive PSP in Table 3. After the order is optimized, *VPL/RV* has higher throughput, and adding the validation and rotation would further increase the throughput, validating the effectiveness of the proposed components. Finally, as the network increases, the advantage of *VPL* over *PostMHL* also increases.

Exp 8: Comparison with MVCC-based Solution. We also implement an MVCC-based variant of the SOTA *PostMHL* by allowing queries to continue accessing the previous index version while a new index version is constructed in the background. Specifically, when a batch of updates arrives, we refresh the graph immediately ($G \rightarrow G'$) and then update the PostMHL in the latest graph snapshot (L'). When queries arrive, we do not wait for the completion of L' maintenance to ensure δt-freshness of query processing, but directly answer queries on the old SP index L that is built on an old snapshot G'. However, since this design violates the *update-first queuing model* and does not distinguish the affected and unaffected queries, some queries may be evaluated using an older index snapshot and produce results that are inconsistent with the latest network state. In our experiments, such results are counted as incorrect according to the δt-freshness requirement. We also compare *VPL* with the MVCC version of *PostMHL* in terms of throughput λ^* and the error ratio of query processing. Because our *VPL* always reports 100% correct results, we only distinguish the error ratio of MVCC: ϵ_u is the ratio during index maintenance, and ϵ_a is the ratio over the entire update interval. As shown in Table 5, although it consistently has a high throughput, during maintenance, nearly half of the queries cannot be answered correctly. Even when we

Table 5: Performance of MVCC

Metric	BJ	ZJ	GD	FLA	WC	EC	UK	CN
Error Ratio ϵ_u	57.6%	48.0%	51.6%	45.6%	46.8%	47.1%	53.9%	31.1%
Error Ratio ϵ_a	6.9%	4.0%	9.2%	8.7%	15.6%	47.0%	32.7%	31.0%
Throughput λ^*	630,176	670,203	619,201	393,476	407,821	391,875	305,213	217,923

amortize to the update interval, the error ratio is still unacceptably high, especially on large networks (ϵ_a is larger than 30% on EC, UK, and CN). Among the systems that guarantee correctness, our VPL has the highest throughput.

9.6 Generation of Queries and Updates Data

We employ taxi trajectories to generate road network updates $\mathcal{U}$ and queries Q, as detailed below.

Trajectory Preprocessing and Filtering: First, we perform spatio-temporal filtering and time slot partitioning on the raw trajectory data to retain valid records: 1) Temporal filtering: Only trajectories within the given time window are preserved; all records outside this period are discarded. 2) Spatial filtering: Trajectories are retained if and only if their origin or destination falls within the predefined geographic latitude-longitude boundary of the target road network. 3) Time slot partitioning: The full day is divided into consecutive, non-overlapping time slots with a fixed interval (e.g., 300 seconds). Each valid trajectory record is assigned to the corresponding time slot based on its timestamp.

Query Generation: We generate realistic path queries Q by matching trajectory data to the underlying road network. 1) Load basic road network attributes, including the travel time of each road segment and the geographic coordinates of network nodes. 2) Parse the start and end road segments of each filtered trajectory. Match the trajectory's origin/destination points to the nearest road network nodes with a spatial error constraint of less than 100 meters. 3) Generate a valid path query for each successfully matched trajectory, representing a real-world travel demand.

Update Generation: Road network updates $\mathcal{U}$ are generated by computing and validating dynamic travel time changes: 1) Outlier removal: For each road segment, sort all observed travel times from trajectories and remove the slowest 5% of records to eliminate biases caused by extreme congestion or abnormal driving behavior. 2) Average travel time computation: Compute the average travel time for each road segment within each time slot. 3) Valid update judgment: An update is generated if the travel time changes by more than 20 seconds, or the relative change exceeds thresholds (an increase of $\geq$100% or a decrease of $\geq$50%) compared with the base travel time or previous update value.